МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Кафедра інформаційних систем та мереж

ЗВІТ

про виконання лабораторної роботи № 8.1

«Пошук та заміна символів у літерному рядку»

(рекурсивний спосіб)

з дисципліни

«Алгоритмізація та програмування»

Студент групи РІ-13
Шах Владислав Ігорьович
Варіант № 32

Прийняв викладач:
Григорович В.Г.

1. Мета роботи та умова завдання

Мета роботи: набути практичних навичок роботи з літерними рядками у мові програмування С++, реалізувати алгоритми обробки масивів символів без використання сторонніх бібліотечних функцій за допомогою рекурсивного підходу.

Умова завдання (Варіант 32):

Дано літерний рядок, який містить послідовність символів s0, ..., sn, ...

1. Підрахувати, скільки разів серед даних символів зустрічається кожний символ, що входить в слово "BASIC".

2. Замінити кожне слово "BASIC" словом "Delphi".

Варіант 32.

Дано літерний рядок, який містить послідовність символів s_0, …, s_n, … .

1. Підрахувати, скільки разів серед даних символів зустрічається кожний символ, що входить в слово “BASIC”.
2. Замінити кожне слово “BASIC” словом “Delphi”.

Варіант 33.

Рис. 1. Умова завдання з методичних вказівок (Варіант 32)

2. Структурна схема програми

Відповідно до вимог, структура програми подана виключно у вигляді структурної схеми зв'язків між функціональними компонентами. Схема відображає ієрархію викликів (стрілки ліворуч) та рекурсивні виклики функцій до самих себе (петлі праворуч).

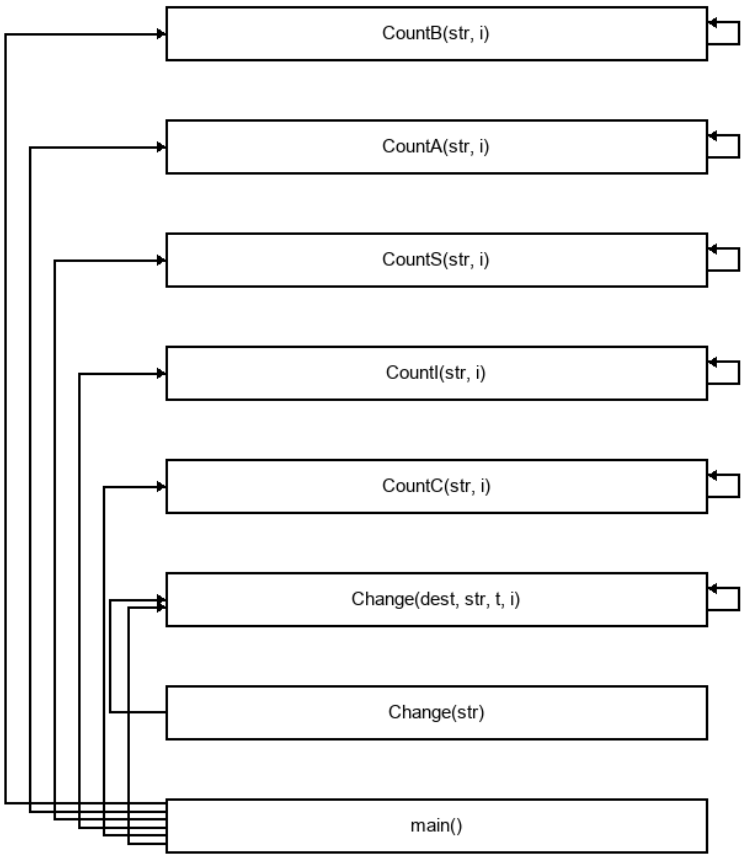


Рис. 2. Структурна схема взаємодії функцій програми

Опис функцій та зв'язків програми:

• main() — головна функція, точка входу в програму. Забезпечує введення початкового рядка, викликає функції обчислення та заміни, а також виводить результати на консоль.

• CountB(str, i) — окрема рекурсивна функція підрахунку кількості літери 'B' у рядку str, починаючи з індексу i.

• CountA(str, i) — окрема рекурсивна функція підрахунку кількості літери 'A' у рядку str.

• CountS(str, i) — окрема рекурсивна функція підрахунку кількості літери 'S' у рядку str.

• CountI(str, i) — окрема рекурсивна функція підрахунку кількості літери 'I' у рядку str.

• CountC(str, i) — окрема рекурсивна функція підрахунку кількості літери 'C' у рядку str.

• Change(dest, str, t, i) — базова рекурсивна функція заміни слова "BASIC" на "Delphi". Забезпечує безпечну роботу з пам'яттю за зразком методички (стор. 102) за допомогою перевірки if (str[i + 4] != 0). У разі збігу слова дописує "Delphi", зміщує вказівники та продовжує рекурсію. Якщо до кінця рядка залишилося менше ніж 5 символів, докопіює залишок рядка без виходу за межі пам'яті.

• Change(str) — перевантажена функція-обгортка, яка динамічно виділяє пам'ять під новий рядок за формулою (int)(len * 6.0 / 5 + 1) + 1 та викликає рекурсивну функцію Change.

3. Текст програми (LABA-8.1.cpp)

```
// LABA-8.1.cpp
// Шах Владислав Ігорьович
// Лабораторна робота № 8.1
// Пошук та заміна символів у літерному рядку
// Варіант 32 (Рекурсивний спосіб)

#ifndef _CRT_SECURE_NO_WARNINGS
#define _CRT_SECURE_NO_WARNINGS
#endif
#include <iostream>
#include <cstring>

using namespace std;

// 1. Окремі рекурсивні функції підрахунку для кожної літери зі слова "BASIC":

// 1.1. Підрахунок кількості літер 'B'
int CountB(const char* str, int i = 0)
{
    if (str[i] != '\0')
    {
        if (str[i] == 'B')
            return 1 + CountB(str, i + 1);
        else
            return CountB(str, i + 1);
    }
    else
        return 0;
}

// 1.2. Підрахунок кількості літер 'A'
int CountA(const char* str, int i = 0)
{
    if (str[i] != '\0')
    {
        if (str[i] == 'A')
            return 1 + CountA(str, i + 1);
        else
            return CountA(str, i + 1);
    }
    else
        return 0;
}

// 1.3. Підрахунок кількості літер 'S'
int CountS(const char* str, int i = 0)
```

```
{
    if (str[i] != '\0')
    {
        if (str[i] == 'S')
            return 1 + CountS(str, i + 1);
        else
            return CountS(str, i + 1);
    }
    else
        return 0;
}

// 1.4. Підрахунок кількості літер 'I'
int CountI(const char* str, int i = 0)
{
    if (str[i] != '\0')
    {
        if (str[i] == 'I')
            return 1 + CountI(str, i + 1);
        else
            return CountI(str, i + 1);
    }
    else
        return 0;
}

// 1.5. Підрахунок кількості літер 'C'
int CountC(const char* str, int i = 0)
{
    if (str[i] != '\0')
    {
        if (str[i] == 'C')
            return 1 + CountC(str, i + 1);
        else
            return CountC(str, i + 1);
    }
    else
        return 0;
}

// 2. Рекурсивна функція заміни слова "BASIC" на "Delphi"
// Безпечна робота з пам'яттю за зразком методички (стор. 102):
// Умова if (str[i + 4] != 0) гарантує, що індекси від i до i+4 існують
// і ми не виходимо за межі рядка.
char* Change(char* dest, const char* str, char* t, int i = 0)
{
    if (str[i + 4] != 0)
    {
        // Перевіряємо всі 5 літер слова "BASIC"
        if (str[i] == 'B' && str[i + 1] == 'A' && str[i + 2] == 'S' &&
            str[i + 3] == 'I' && str[i + 4] == 'C')
        {
            strcat(t, "Delphi");
            return Change(dest, str, t + 6, i + 5);
        }
        else
        {
            *t++ = str[i++];
            *t = '\0';
```

```
            return Change(dest, str, t, i);
        }
    }
    else
    {
        // Якщо до кінця рядка залишилося менше ніж 5 символів,
        // слово "BASIC" поміститися вже не може.
        // Докопіюємо залишкові символи до кінця рядка ('\0').
        if (str[i] != '\0')
        {
            *t++ = str[i++];
            *t = '\0';
            return Change(dest, str, t, i);
        }
        else
        {
            *t = '\0';
            return dest;
        }
    }
}

// Зручна обгортка з виділенням пам'яті за формулою з методички (стор. 100):
// (int)(len * 6.0 / 5 + 1) + 1
// де 6.0 - довжина нового слова "Delphi", 5 - довжина старого слова "BASIC"
char* Change(const char* str)
{
    size_t len = strlen(str);
    char* dest = new char[(int)(len * 6.0 / 5 + 1) + 1];
    dest[0] = '\0';
    return Change(dest, str, dest, 0);
}

#ifndef UNIT_TESTING
int main()
{
    char str[101];
    cout << "Enter string:" << endl;
    cin.getline(str, 100);

    cout << "\n--- Lab 8.1 (Recursive) Variant 32 ---" << endl;
    cout << "Initial string: " << str << endl;

    // 1. Окремий підрахунок для КОЖНОЇ літери зі слова "BASIC":
    int countB = CountB(str, 0);
    int countA = CountA(str, 0);
    int countS = CountS(str, 0);
    int countI = CountI(str, 0);
    int countC = CountC(str, 0);
    int total = countB + countA + countS + countI + countC;

    cout << "\n1. Occurrences of each character from 'BASIC':" << endl;
    cout << "   'B': " << countB << endl;
    cout << "   'A': " << countA << endl;
    cout << "   'S': " << countS << endl;
    cout << "   'I': " << countI << endl;
    cout << "   'C': " << countC << endl;
    cout << "   Total ('B','A','S','I','C'): " << total << endl;
```

```
    // 2. Заміна "BASIC" на "Delphi"
    // Виділення динамічної пам'яті за формулою методички:
    // довжина "BASIC" = 5, довжина "Delphi" = 6 -> коефіцієнт 6.0 / 5
    size_t len = strlen(str);
    char* dest1 = new char[(int)(len * 6.0 / 5 + 1) + 1];
    dest1[0] = '\0';

    char* dest2 = Change(dest1, str, dest1, 0);

    cout << "\n2. Modified string (param) : " << dest1 << endl;
    cout << "   Modified string (result): " << dest2 << endl;

    delete[] dest1;
    return 0;
}
#endif
```

4. Модульне тестування (UnitTest8.1)

Для перевірки коректності функціонування кожної окремої рекурсивної функції розроблено набір юніт-тестів за допомогою Microsoft Native Unit Test Framework.

```
#include "pch.h"
#include "CppUnitTest.h"

#define UNIT_TESTING
#include "../LABA-8.1/LABA-8.1.cpp"

using namespace Microsoft::VisualStudio::CppUnitTestFramework;

namespace UnitTest81
{
    TEST_CLASS(UnitTest81)
    {
    public:
        TEST_METHOD(TestCountEachLetter)
        {
            const char* str = "BASIC and BASIC";
            // Кожна окрема функція підраховує відповідну букву зі слова BASIC
            Assert::AreEqual(2, CountB(str, 0));
            Assert::AreEqual(2, CountA(str, 0));
            Assert::AreEqual(2, CountS(str, 0));
            Assert::AreEqual(2, CountI(str, 0));
            Assert::AreEqual(2, CountC(str, 0));
        }

        TEST_METHOD(TestCountZeroOccurrences)
        {
            const char* str = "HELLO WORLD 123";
            Assert::AreEqual(0, CountB(str, 0));
            Assert::AreEqual(0, CountA(str, 0));
            Assert::AreEqual(0, CountS(str, 0));
            Assert::AreEqual(0, CountI(str, 0));
            Assert::AreEqual(0, CountC(str, 0));
        }
```

```
        TEST_METHOD(TestChangeNormal)
        {
            char* res1 = Change("I like BASIC language");
            Assert::AreEqual("I like Delphi language", res1);
            delete[] res1;

            char* res2 = Change("BASIC and BASIC");
            Assert::AreEqual("Delphi and Delphi", res2);
            delete[] res2;

            char* res3 = Change("BASIC");
            Assert::AreEqual("Delphi", res3);
            delete[] res3;
        }

        TEST_METHOD(TestChangeShortStrings)
        {
            // Тести на рядки, коротші за 5 символів, щоб гарантувати відсутність виходу
за межі пам'яті
            char* res1 = Change("BAS");
            Assert::AreEqual("BAS", res1);
            delete[] res1;

            char* res2 = Change("B");
            Assert::AreEqual("B", res2);
            delete[] res2;

            char* res3 = Change("");
            Assert::AreEqual("", res3);
            delete[] res3;

            char* res4 = Change("No words");
            Assert::AreEqual("No words", res4);
            delete[] res4;
        }
    };
}
```

Результати автоматизованого виконання юніт-тестів (усі 4 тести пройшли успішно, зокрема перевірено стійкість до коротких рядків):

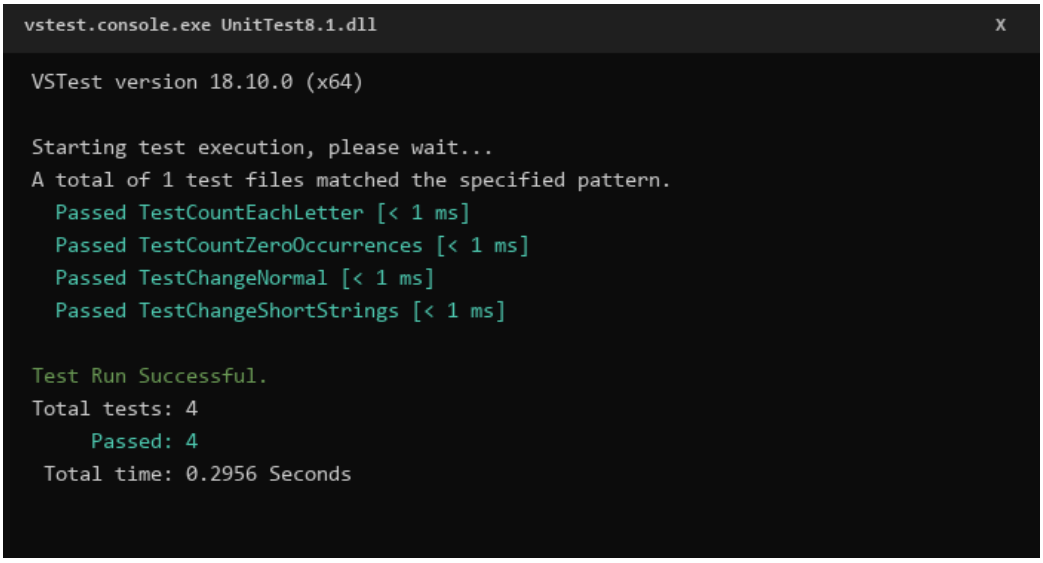


Рис. 3. Результати проходження unit-тестів (vstest.console.exe)

5. Результати виконання програми

Консольне виконання програми з тестовим вхідним рядком "Hello BASIC world! BASIC rocks.":

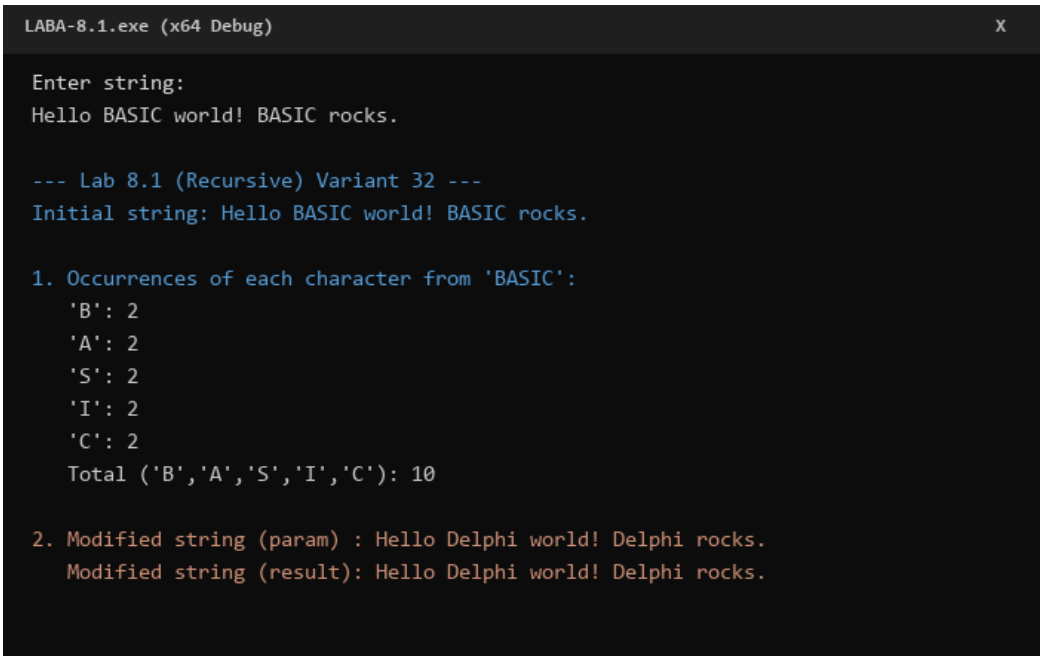


Рис. 4. Консольне вікно виконання програми

6. Посилання на GitHub репозиторій

Вихідний код лабораторної роботи, проектні файли Visual Studio, юніт-тести та звіт завантажено до віддаленого репозиторію на GitHub:

https://github.com/vivjk686/LABA-8.1

7. Висновки

Під час виконання лабораторної роботи № 8.1 було досліджено та практично реалізовано алгоритми рекурсивної посимвольної обробки літерних рядків (масивів символів char[]) мовою C++.

Було вирішено обидві задачі варіанта № 32: для кожної окремої букви слова "BASIC" ('B', 'A', 'S', 'I', 'C') реалізовано окрему рекурсивну функцію підрахунку, а також реалізовано безпечну заміну слова "BASIC" на "Delphi" з безпечною обробкою меж пам'яті (if (str[i + 4] != 0)) та коректним виділенням динамічної пам'яті. Усі функції протестовано unit-тестами, побудовано структурну схему та оформлено звіт.